1. Run the buggy application .py in the repo assigned

```
PS C:\Users\Shriranjini\Downloads\Lab 7> python -m pip show pytest
Author-email:
License: MIT
Location: C:\Users\Shriranjini\anaconda3\Lib\site-packages
Requires: colorama, iniconfig, packaging, pluggy
Required-by:
PS C:\Users\Shriranjini\Downloads\Lab 7> python pizza_order.py
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): cheese,olives
Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): M
Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): 900/3
Per Person: 300.0
```

2. Complete the code in test_processor.py using hypothesis, Run the test_processor.py

```
=========================================================== short test summary info ============================================================
FAILED test_pizza_order.py::test_toppings_price_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_pizza_order.py::test_pizza_inches_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_pizza_order.py::test_split_bill_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
============================================================ 3 failed, 2 passed in 3.68s ============================================================
PS C:\Users\Shriranjini\Downloads\Lab 7>
```

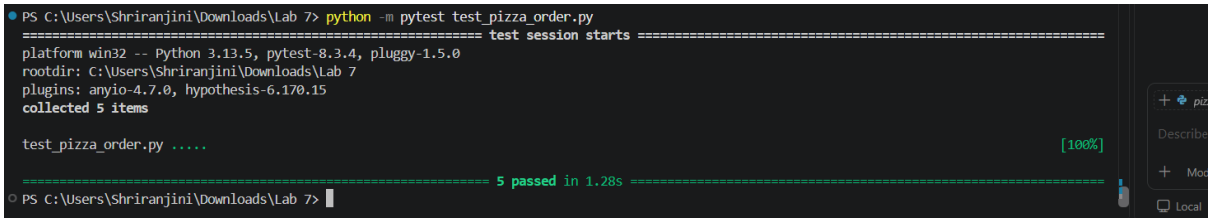
3. Fix the buggy application python script:

```
PS C:\Users\Shriranjini\Downloads\Lab 7> python pizza_order.py
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): cheese,olives
Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): M
Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): 900/3
Per Person: 300.0
```

4. Re-Run the test_processor.py wrt to fixed processor and take the ss of the output



Fuzz testing using the Hypothesis library helped identify input validation bugs in the original pizza ordering application. After fixing the bugs using defensive programming, all five test cases passed successfully.

test_pizza_order.py

```
from hypothesis import given, strategies as st

from pizza_order import toppings_price, pizza_inches, split_bill

# Test 1: Fuzz testing toppings_price()

@given(st.one_of(st.text(), st.none()))

def test_toppings_price_no_crash(s):
```

```
    result = toppings_price(s)
    assert result is None or isinstance(result, int)

# Test 2: Fuzz testing pizza_inches()
@given(st.one_of(st.text(), st.none()))
def test_pizza_inches_no_crash(s):
    result = pizza_inches(s)
    assert result is None or isinstance(result, int)

# Test 3: Fuzz testing split_bill()
@given(st.one_of(st.text(), st.none()))
def test_split_bill_no_crash(s):
    result = split_bill(s)
    assert result is None or isinstance(result, float)

# Test valid topping combinations
@given(st.lists(
    st.sampled_from(["cheese", "olives", "onion", "paneer"]),
    min_size=1
))
def test_valid_toppings(toppings):
    result = toppings_price(",".join(toppings))
    expected = sum(
        {"cheese": 40, "olives": 30, "onion": 20, "paneer": 50}[t]
        for t in toppings
    )
    assert result == expected
```

```
# Test valid pizza sizes
@given(st.sampled_from(["S", "M", "L", "s", "m", "l"]))
def test_valid_pizza_sizes(size):
    assert pizza_inches(size) in [8, 10, 12]
```